

微服务SpringCloudAlibaba

授课目标

- 目标01-掌握网关的基本概念Route、Predicate、Filter
- 目标02-掌握网关的静态路由与动态路由配置
- 目标03-理解网关工作原理
- 目标04-掌握网关常见的断言规则：Path、Header、Datetime、Cookie
- 目标05-掌握网关过滤器使用：添加前缀、去除前缀、路径重写等
- 目标06-掌握网关限流、熔断、鉴权、跨域配置等常见场景实现方案

第5章 微服务网关

5.1 网关简介

上一章内容我们介绍通过Spring Cloud LoadBalancer实现了微服务之间的调用和负载均衡，以及使用Spring Cloud OpenFeign声明式调用，**那我们的各种微服务又要如何提供给外部应用调用呢？**

直接暴露微服务API接口会存在以下问题：

- 客户端会多次请求不同的微服务，增加了客户端的复杂性。
- 存在跨域请求，在一定场景下处理会变得相对比较复杂。
- 实现认证复杂，每个微服务都需要独立认证。
- 难以重构，项目迭代可能导致微服务重新划分。如果客户端直接与微服务通讯，那么重构将会很难实施。
- 如果某些微服务使用了防火墙、浏览器不友好的协议，直接访问会有一定困难。

面对类似上面的问题，我们要如何解决呢？

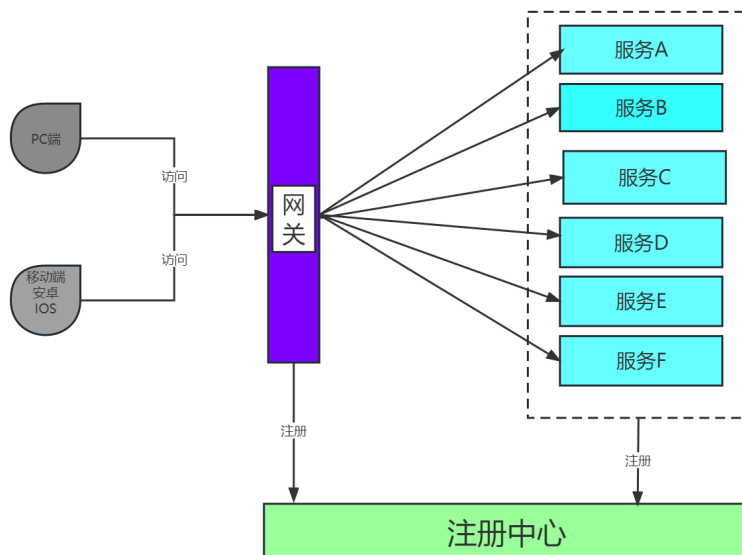
答案就是：服务网关！在微服务系统中微服务资源不会直接暴露在公网。

网关的意义具体体现在下面几个方面：

- 网关可以做一些身份认证、权限管理、防止非法请求操作服务等，对服务起一定保护作用。
- 网关将所有微服务统一管理，对外统一暴露，外界系统不需要知道微服务架构个服务相互调用的复杂性，同时也避免了内部服务一些敏感信息泄露问题。
- 易于监控。可在微服务网关收集监控数据并将其推送到外部系统进行分析。
- 客户端只跟服务网关打交道，减少了客户端与各个微服务之间的交互次数。
- 多渠道支持，可以根据不同客户端（WEB端、移动端、桌面端...）提供不同的API服务网关。
- 网关可以用来做流量监控。在高并发下，对服务限流、降级。
- 网关把服务从内部分离出来，方便测试。

微服务网关能够实现，路由、负载均衡等多种功能。类似Nginx反向代理的功能。在微服务架构中，后端服务往往不直接开放给调用端，而是通过一个API网关根据请求的URL，路由到相应的服务。当添加API网关后，在第三方调用端和服务提供方之间就创建了一面墙，在API网关中进行权限控制，同时API网关将请求以负载均衡的方式发送给后端服务。

在微服务架构系统下，网关是系统唯一对外的入口，介于客户端与服务器端之间，用于对请求进行鉴权、限流、**路由**、监控等功能。**本身也是一个微服务。**



总结：网关微服务是所有微服务的入口。主要用于实现鉴权、限流、路由、监控【监控所有方法执行调用时间】等功能...

5.1.1 Gateway简介

SpringCloud Gateway项目基于 Spring 5.0, Spring Boot 2.0 和 Project Reactor 等技术开发的网关, 它旨在为微服务架构提供一种简单有效的统一的 API 路由管理方式。

SpringCloud Gateway 作为 Spring Cloud 生态系统中的网关, 目标是替代 Zuul, 在Spring Cloud 2.0 以上版本中, 没有对新版本的Zuul 2.0以上最新高性能版本进行集成, 仍然还是使用的Zuul 2.0之前的非Reactor模式的老版本。而为了提升网关的性能, SpringCloud Gateway是基于WebFlux框架实现的, 而WebFlux框架底层则使用了**高性能通信框架Netty**。

Spring Cloud Gateway 的目标, 不仅提供统一的路由方式, 并且基于 Filter 链的方式提供了网关基本的功能, 例如: 安全, 监控/指标, 和限流。

SpringCloud官方对Gateway 特征介绍如下:

- 基于 Spring Framework 5, Project Reactor 和 Spring Boot 2.0
- 集成 Spring Cloud DiscoveryClient
- Predicates 和 Filters 作用于特定路由, 易于编写的 Predicates 和 Filters
- 具备一些网关的高级功能: 动态路由、限流、路径重写
- 集成Spring Cloud DiscoveryClient
- 集成熔断器CircuitBreaker

从以上的特征来说, 和Zuul的特征差别不大。SpringCloud Gateway和Zuul主要的区别, 还是在底层的通信框架上。

5.1.2 三个重要概念

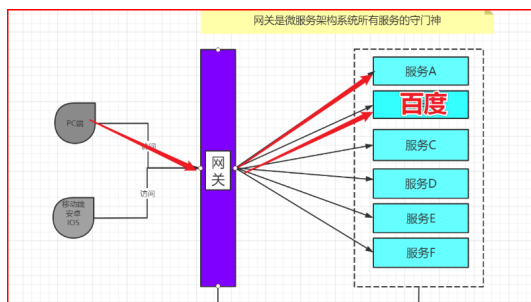
在 Spring Cloud Gateway 中有三个非常重要的概念：

1. **route路由**：路由是网关的最基本组成，由一个路由 id、一个目标地址 url，一组断言工厂及一组 filter 组成。若断言为 true，则请求将经由 filter 被路由到目标 url。
2. **predicate断言**：这是 Java8 的 Predicate，可以使用它来匹配来自 HTTP 请求的任何内容。所谓断言其实理解为条件判断更合适，根据当前的 http 请求进行指定规则的匹配，比如说 http 请求头，请求时间等。只有当匹配上规则时，断言才为 true，此时请求才会被直接路由到目标地址，或先路由到某过滤器链，经过过滤器链的层层处理后，再路由到相应的目标地址。
3. **filter过滤器**：对请求进行处理的逻辑部分。当请求的断言为 true 时，会被路由到设置好的过滤器，以对请求进行处理。例如，可以为请求添加一个请求头，或添加一个请求参数，或对请求 URI 进行修改等。总之，就是对请求进行处理。

5.1.3 与Zuul的对比

1. Zuul 闭源：Spring Cloud Gateway 是 Zuul 网关的替代者。之所以弃用 Zuul 并不是因为 Zuul 在功能有什么大的问题。而是因为最开始的 Zuul 是开源的，所以 Spring Cloud 就集成了 Zuul 做网关。但后来 Zuul 又宣布闭源，所以 Spring Cloud 自己开发了 Spring Cloud Gateway。再后来 Zuul 2.0 又开源了，但 Spring Cloud 不再集成它了。
2. Zuul：Zuul 是由 Netflix 开源的 API 网关，基于 servlet 的，使用阻塞 API，不支持长连接。
3. Spring Cloud Gateway：Spring Cloud Gateway 是 Spring Cloud 自己开发的开源 API 网关，建立在 Spring5，Reactor 和 Spring Boot 2 之上，使用非阻塞 API，异步。

5.2 网关案例



5.2.1 案例：静态路由

需求：用户访问网关，直接跳转到百度

1、配置路由到服务

目标：搭建网关微服务 05-gateway-config，并且配置路由到服务

1) 创建工程

复制工程 02-order-nacos，并重命名为 05-gateway-config。

2) 修改pom文件

仅保留 Actuator 依赖，并添加 spring cloud gateway 依赖。

```
1  <!--gateway 依赖-->
2  <dependency>
3      <groupId>org.springframework.cloud</groupId>
4      <artifactId>spring-cloud-starter-gateway</artifactId>
5  </dependency>
6
7  <dependency>
8      <groupId>org.springframework.boot</groupId>
9      <artifactId>spring-boot-devtools</artifactId>
10     <scope>runtime</scope>
11     <optional>true</optional>
12 </dependency>
13 <dependency>
14     <groupId>org.springframework.boot</groupId>
15     <artifactId>spring-boot-starter-test</artifactId>
16     <scope>test</scope>
17 </dependency>
```

3) 修改代码类

代码中，只需一个启动类，其它全部删除。

```
1  @SpringBootApplication
2  public class GatewayApplication {
3      public static void main(String[] args) {
4          SpringApplication.run(GatewayApplication.class, args);
5      }
6  }
```

4) 修改配置文件

```
1  server:
2      port: 9000
3  spring:
4      cloud:
5          gateway:
6              routes:
7                  ## 路由id
8                  - id: baidu_route
9                  ## 路由uri地址
10                 uri: https://www.baidu.com
11                 predicates: #断言就是路由的规则
12                 ## 拦截所有请求
13                 - Path=/**
```

5) 测试



2、配置路由到百度

1) 创建工程

复制工程 05-gateway-config，并重命名为 05-gateway-api。

2) 修改配置文件

去掉原来配置的路由策略，仅剩如下内容。

```
1 | server.port: 9001
```

3) 修改启动类

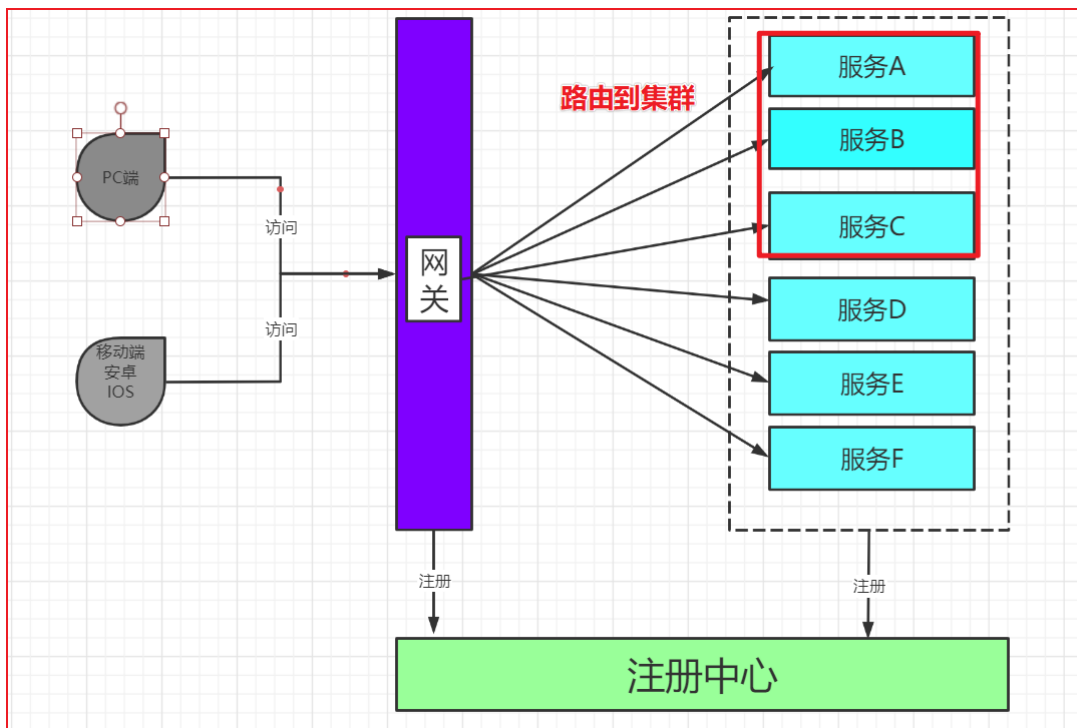
在启动类中添加一个@Bean 方法，用于设置路由策略。

```
1 | @SpringBootApplication//启动引导类，同时也是配置类
2 | public class GatewayApiApplication {
3 |     public static void main(String[] args) {
4 |         SpringApplication.run(GatewayApiApplication.class, args);
5 |     }
6 |     //配置路由规则
7 |     @Bean
8 |     public RouteLocator someRouteLocator(RouteLocatorBuilder builder){
9 |         return builder.routes().route("baidu_route",predicateSpec ->
10 |             predicateSpec
11 |                 .path("/**")
12 |                 .uri("https://www.baidu.com")).build();
13 |     }
14 | }
```

4) 测试



5.2.2 案例：动态路由\负载均衡



根据微服务名称进行 负载均衡。有一个微服务，有三个提供者，这里要通过 spring cloud gateway 实现对该微服务的负载均衡访问。

1、方式1-配置式-配置文件

1) 创建工程

复制工程 05-gateway-config，并重命名为 05-gateway-config-loadbalancer。

2) 添加依赖

```
1 <!-- nacos discovery 依赖 -->
2 <dependency>
3     <groupId>com.alibaba.cloud</groupId>
4     <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
5 </dependency>
6 <!-- 负载均衡器 -->
7 <dependency>
8     <groupId>org.springframework.cloud</groupId>
9     <artifactId>spring-cloud-starter-loadbalancer</artifactId>
10 </dependency>
```

3) 修改配置文件

```
1 server:
2     # 默认端口是8080
3     port: 9000
4 spring:
5     application:
6         name: msc-gateway
```

```

7   cloud:
8     nacos:
9       # 注册中心地址
10      discovery:
11        server-addr: 127.0.0.1:8848
12      gateway:
13        discovery:
14          locator:
15            # 配置开启与DiscoveryClient整合
16            enabled: true
17      routes:
18        # 路由id
19        - id: ribbon_route
20          # 路由uri地址, loadbalancer://服务名称 【DNS域名解析的思路】
21          uri: lb://msc-payment
22          predicates:
23            # 拦截所有提供者的请求到提供者集群服务
24            - Path=/payment/depart/**
25

```

4) 测试



2、方式2-API式-配置类

1) 创建工程

复制工程 05-gateway-api, 并重命名为 05-gateway-api-loadbalancer。

2) 添加依赖

由于要注册到 Nacos, 所以需要导入 Nacos Discovery 依赖。

```

1  <!--nacos discovery 依赖-->
2  <dependency>
3    <groupId>com.alibaba.cloud</groupId>
4    <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
5  </dependency>
6  <dependency>
7    <groupId>org.springframework.cloud</groupId>
8    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
9  </dependency>

```

3) 修改配置文件

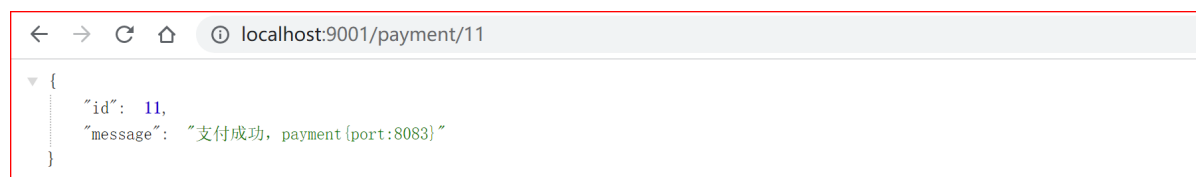
```
1 server:
2   port: 9001
3 spring:
4   application:
5     name: msc-gateway
6   cloud:
7     nacos:
8       # 注册中心地址
9     discovery:
10      server-addr: 127.0.0.1:8848
```

4) 修改启动类

在启动类中添加如下@Bean 方法。

```
1 @SpringBootApplication
2 public class GatewayApplication {
3
4     public static void main(String[] args) {
5         SpringApplication.run(GatewayApplication.class, args);
6     }
7     //配置路由规则
8     @Bean
9     public RouteLocator someRouteLocator(RouteLocatorBuilder builder){
10        //路由构建器对象，构建一个路由规则
11        return builder.routes().route("loadbalancer_route",predicateSpec ->
12            predicateSpec
13                .path("/payment/**")
14                .uri("lb://msc-payment")).build();
15    }
```

4) 测试



扩展: PathMatcher

<https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/#the-path-route-predicate-factory>

<https://docs.spring.io/spring-framework/docs/5.2.12.RELEASE/javadoc-api/>

`PathMatcher` 实现了Ant路径匹配风格。借鉴了ApacheAnt项目的大部分路径匹配代码实现。

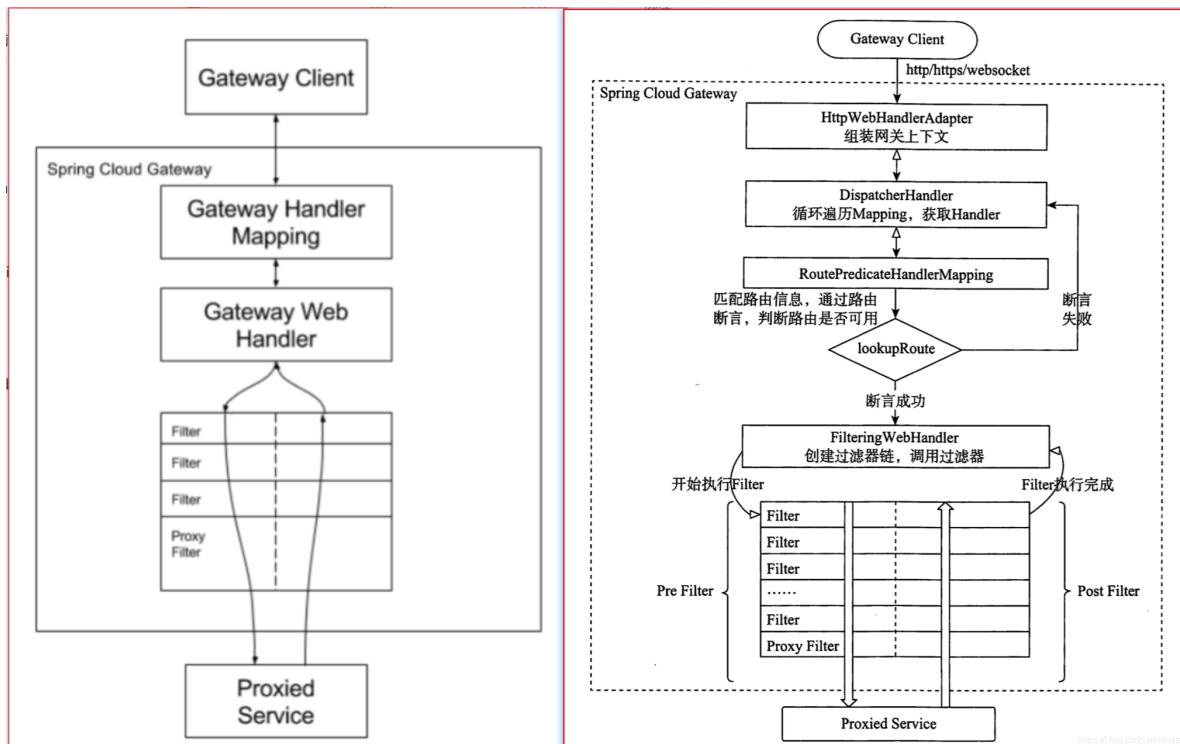
规则如下:

- `?` 只匹配正斜杠 / 路径后一个字符
- `*` 匹配正斜杠 / 路径后的多个字符串
- `**` 匹配正斜杠 / 路径后的多个正斜杠内的多个字符串
- `{spring:[a-z]+}` 匹配正则表达式 `[a-z]+` , 并将匹配到的值赋值给spring变量

5.3 Gateway工作原理

<https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/>

如下图，左边为官方提供High Level 流程图，右边详细流程图。



网关两端，三大件：客户端Client，服务端Proxied Service

- 网关处理器映射器组件HandlerMapping
- 网关web处理器组件Web Handler
- 网关过滤器组件 Filter

主要工作原理：

1. Gateway的客户端向Spring Cloud Gateway发起请求
2. 首先请求会HttpWebHandlerAdapter进行提取组装成网关的Context，然后网关的Context会传递到DispatcherHandler。
3. DispatcherHandler是所有请求的分发处理器，**DispatcherHandler主要负责分发请求对应的处理器**，比如将请求分发到对应RoutePredicateHandlerMapping(路由断言处理器映射器)。
4. **RoutePredicateHandlerMapping路由断言处理映射器主要用于路由的查找**，以及找到路由后返回对应的FilteringWebHandler，如果没有找到则丢弃处理。
5. **FilteringWebHandler主要负责组装Filter链表**并调用Filter执行一系列Filter处理
6. 然后把请求转到后端对应的**代理服务【Proxied Service】**处理，处理完毕后，将Response返回到Gateway客户端。

在Filter链中，通过虚线分割Filter的原因是，过滤器可以在转发请求之前处理，或者在代理服务返回结果之后处理。

- PreFilter前置过滤器：所有的Pre类型的Filter执行完毕之后，才会转发请求到被代理的服务处理。
- PostFilter后置过滤器：被代理的服务把所有请求完毕之后，才会执行Post类型的过滤器。

5.4 网关路由断言工厂

5.4.1 什么是Predicate Factories 断言工厂？

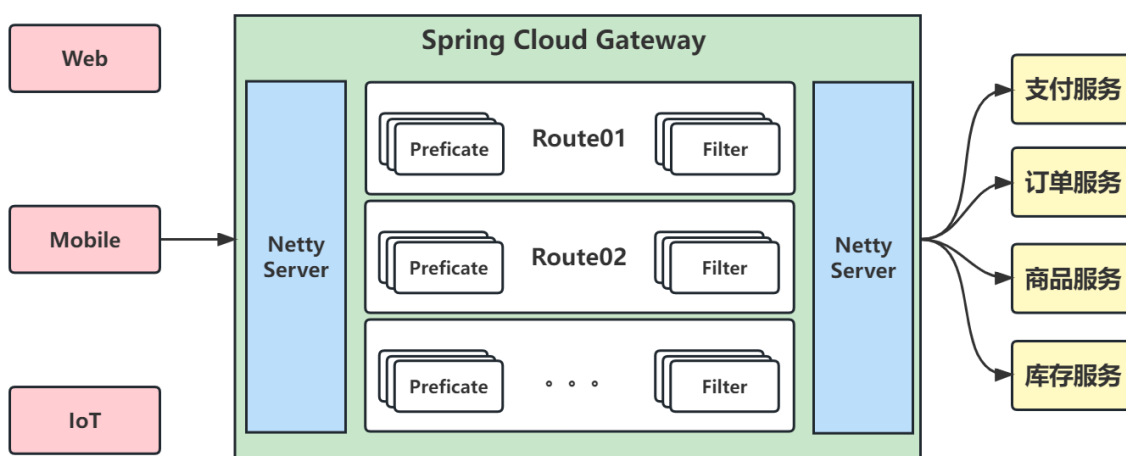
路由匹配规则

Spring Cloud Gateway的主要功能之一是转发请求，转发规则的定义主要包含三个部分

- **Route (路由)**：路由是网关的基本单元，由ID、URI、一组Predicate、一组Filter组成，根据Predicate进行匹配转发。
- **Predicate (谓词、断言)**：路由转发的判断条件，目前SpringCloud Gateway支持多种方式，常见如：Path、Query、Method、Header等，写法必须遵循 key=value的形式
- **Filter (过滤器)**：过滤器是路由转发请求时所经过的过滤逻辑，可用于修改请求、响应内容

Spring Cloud Gateway 的功能很强大，我们仅仅通过 Predicates 的设计就可以看出来，前面我们只是使用了 predicates 进行了简单的条件匹配，其实 Spring Cloud Gateway 帮我们内置了很多 Predicates 功能。

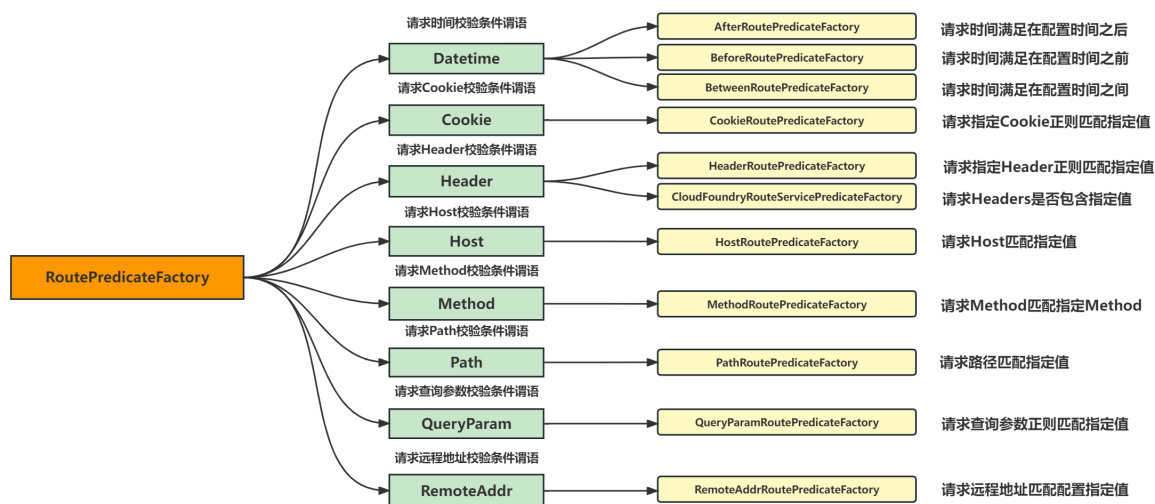
Spring Cloud Gateway 是通过 Spring WebFlux 的 HandlerMapping 做为底层支持来匹配到转发路由，Spring Cloud Gateway 内置了很多 Predicates 工厂，这些 Predicates 工厂通过不同的 HTTP 请求参数来匹配，多个 Predicates 工厂可以组合使用。



5.4.2 断言条件总体介绍

Predicate 来源于 Java 8，是 Java 8 中引入的一个函数，Predicate 接受一个输入参数，返回一个布尔值结果。该接口包含多种默认方法来将 Predicate 组合成其他复杂的逻辑（比如：与，或，非）。可以用于接口请求参数校验、判断新老数据是否有变化需要进行更新操作。

在 Spring Cloud Gateway 中 Spring 利用 Predicate 的特性实现了各种路由匹配规则，有通过 Header、请求参数等不同的条件来进行作为条件匹配到对应的路由。网上有一张图总结了 Spring Cloud 内置的几种 Predicate 的实现。



说白了 Predicate 就是为了实现一组匹配规则，方便让请求过来找到对应的 Route 进行处理，接下来我们接下 Spring Cloud GateWay 内置几种 Predicate 的使用。

规则	实例	说明
Path	- Path=/gate/,/rule/	当请求的路径为gate、rule开头的时，转发到 http://localhost:9001 服务器上
Before	- Before=2022-01-20T17:42:47.789-07:00[America/Denver]	在某个时间之前的请求才会被转发到 http://localhost:9001 服务器上
After	- After=2022-01-20T17:42:47.789-07:00[America/Denver]	在某个时间之后的请求才会被转发
Between	- Between=2022-01-20T17:42:47.789-07:00[America/Denver],2017-01-21T17:42:47.789-07:00[America/Denver]	在某个时间段之间的才会被转发

规则	实例	说明
Cookie	- Cookie=chocolate, ch.p	名为chocolate的表单或者满足正则ch.p的表单才会被匹配到进行请求转发
Header	- Header=X-Request-Id, \d+	携带参数X-Request-Id或者满足\d+的请求头才会匹配
Host	- Host= www.hero123.com	当主机名为 www.hero123.com 的时候直接转发到 http://localhost:9001 服务器上
Method	- Method=GET	只有GET方法才会匹配转发请求，还可以限定POST、PUT等请求方式

5.4.3 Path路由断言工厂

路由断言工厂的使用方式有两种：yaml配置文件式与 API 配置类式。

以下讲义中的代码：

- yaml配置文件式：在05-gateway-config 工程上直接进行修改，即yaml配置文件。
- API 配置类式：在 05-gateway-api 工程上直接进行修改，即启动类中设置。

1) 规则

该断言工厂用于，判断请求路径中是否包含指定的uri。若包含，则匹配成功，断言为true，此时会将该匹配上的 uri 拼接到要转向的目标 uri 的后面，形成一个统一的 uri。

2) 配置文件

添加了两个路由策略。

```

1      # 配置Path断言工厂1，路由到payment
2      - id: path_payment_route
3        uri: http://localhost:8081
4        predicates:
5          - Path=/payment/**
6      # 配置Path断言工厂2，路由到order
7      - id: path_order_route
8        uri: http://localhost:8080
9        predicates:
10         - Path=/order/**

```

3) 配置类

直接修改路由方法，添加路由策略

```

1 //Path配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("path_payment_route", ps -> ps.path("/payment/**")
6             .uri("http://localhost:8081"))
7         .route("path_order_route", ps -> ps.path("/order/**")
8             .uri("http://localhost:8080"))
9         .build();
10 }

```

5.4.4 路由优先级

可以通过配置order指定路由优先级，order越小优先级越高。

以下代码执行<http://localhost:9000>，路由到<https://www.baidu.com>，如果不写order，则会路由到<https://cn.bing.com/>

```

1 #路由优先级
2 - id: bing_route
3   uri: https://cn.bing.com/
4   predicates:
5     - Path=/**
6 - id: baidu_route
7   uri: https://www.baidu.com/
8   predicates:
9     - Path=/**

```

5.4.5 After路由断言工厂

1) 规则

After断言工厂的参数是一个 UTC 格式的时间。其会将请求访问到 Gateway 的时间与该参数时间对比。若请求时间在参数时间之后，则匹配成功，断言为 true。

2) 配置文件

```

1 # after断言工厂
2 - id: after_route
3   uri: https://www.baidu.com
4   predicates:
5     - After=2022-10-20T17:42:47.789-07:00[Asia/Shanghai]

```

3) 配置类

```

1 //After配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     //ISO-8601日历系统中带有时区的日期时间
5     ZonedDateTime dateTime =
6     LocalDateTime.now().minusDays(5).atZone(ZoneId.systemDefault());
7     return builder.routes()
8         .route("after_route", ps -> ps.after(dateTime)//设置after路由断言
9             .uri("https://www.baidu.com"))
10        .build();

```

5.4.6 扩展了解断言工厂

1、Before路由断言工厂

1) 规则

Before断言工厂的参数是一个 UTC 格式的时间。其会将请求访问到 Gateway 的时间与该参数时间相比，若请求时间在参数时间之前，则匹配成功，断言为 true。

2) 配置文件

```

1 # Before断言工厂
2 - id: Before_route
3   uri: https://www.baidu.com
4   predicates:
5     - Before=2023-01-20T17:42:47.789-07:00[Asia/Shanghai]

```

3) 配置类

```

1 //Before配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     //ISO-8601日历系统中带有时区的日期时间
5     ZonedDateTime dateTime =
6     LocalDateTime.now().minusDays(5).atZone(ZoneId.systemDefault());
7     return builder.routes()
8         .route("before_route", ps -> ps.before(dateTime)//设置before路由断言
9             .uri("https://www.baidu.com"))
10        .build();

```

2、Between路由断言工厂

1) 规则

该断言工厂的参数是两个 UTC 格式的时间。其会将请求访问到 Gateway 的时间与这两个参数时间相比，若请求时间在这两个参数时间之间，则匹配成功，断言为 true。

2) 配置文件

```
1      #Between断言工厂
2      - id: Between_route
3        uri: https://www.baidu.com
4      predicates:
5        - Between=2022-09-20T17:42:47.789-07:00[Asia/Shanghai],2023-01-
          20T17:42:47.789-07:00[Asia/Shanghai]
```

3) 配置类

直接修改路由方法。

```
1  //Between配置路由规则
2  @Bean
3  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4      //ISO-8601日历系统中带有时区的日期时间
5      ZonedDateTime minusTime =
LocalDateTime.now().minusDays(5).atZone(ZoneId.systemDefault());
6      ZonedDateTime plusTime =
LocalDateTime.now().plusDays(3).atZone(ZoneId.systemDefault());
7      return builder.routes()
8          .route(ps -> ps.between(minusTime, plusTime)//设置before路由断言
9              .uri("https://www.baidu.com")
10             .id("between_route"))
11          .build();
12  }
```

3、Cookie路由断言工厂

1) 规则

该断言工厂中包含两个参数，分别是 cookie 的 key 与 value。当请求中携带了指定 key 与 value 的 cookie 时，匹配成功，断言为 true。

2) 配置文件

```
1      #Cookie断言工厂
2      - id: cookie_route
3        uri: https://www.baidu.com
4      predicates:
5        - Cookie=love,beauty
```

3) 配置类

直接修改路由方法。

```
1 //cookie配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("cookie_route", ps -> ps.cookie("love", "beauty")
6             .uri("https://www.baidu.com"))
7         .build();
8 }
```

4、Header路由断言工厂

1) 规则

该断言工厂中包含两个参数，分别是请求头 header 的 key 与 value。当请求中携带了指定 key 与 value 的 header 时，匹配成功，断言为 true。

2) 配置文件

```
1 #Header断言工厂
2 - id: header_route
3   uri: https://www.baidu.com
4   predicates:
5     - Header=X-Request-Id, \d+
```

3) 配置类

直接修改路由方法。

```
1 //header配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("header_route", ps -> ps.header("X-Request-Id", "\\d+")
6             .uri("https://www.baidu.com"))
7         .build();
8 }
```

5、Host路由断言工厂

1) 规则

该断言工厂中包含的参数是请求头中的 Host 属性。当请求中携带了指定的 Host 属性值时，匹配成功，断言为 true。

2) 修改hosts文件

修改 C:\Windows\System32\drivers\etc 中的 hosts 文件，为 127.0.0.1 这个 ip 指定多个主机名。例如，在该文件中添加如下内容：

3) 配置文件

```
1      #Host断言工厂
2      - id: host_route
3        uri: https://www.baidu.com
4        predicates:
5          - Host=mylocalhost:9000, myhost:9000
```

4) 配置类

直接修改路由方法。

```
1  //host配置路由规则
2  @Bean
3  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4      return builder.routes()
5          .route("host_route", ps -> ps.host("mylocal:9001")
6              .uri("https://www.baidu.com"))
7          .build();
8  }
```

6、Method路由断言工厂

1) 规则

该断言工厂用于判断请求是否使用了指定的请求方法，是 POST，还是 GET 等。当请求中使用了指定的请求方法时，匹配成功，断言为 true。

2) 配置文件

```
1      #Method断言工厂
2      - id: method_route
3        uri: https://www.baidu.com
4        predicates:
5          - Method=GET,POST
```

3) 配置类

直接修改路由方法。

```
1  //method配置路由规则
2  @Bean
3  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4      return builder.routes()
5          .route("method_route", ps -> ps.method("POST")
6              .uri("https://www.baidu.com"))
7          .build();
8  }
```

7、Query路由断言工厂

1) 规则

该断言工厂用于从请求中查找指定的请求参数。其可以只查看参数名称，也可以同时查看参数名与参数值。当请求中包含了指定的参数名，或名值对时，匹配成功，断言为 true。

2) 配置文件

```
1      #Query断言工厂
2      - id: query_route
3        uri: https://www.baidu.com
4        predicates:
5          - Query=name,hero
6          - Query=age,18
```

3) 配置类

直接修改路由方法。

```
1      //query配置路由规则
2      @Bean
3      public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4          return builder.routes()
5              .route("query_route", ps -> ps.query("name", "hero")
6                  .and()
7                  .query("age", "18")
8                  .uri("https://www.baidu.com"))
9              .build();
10     }
```

8、RemoteAddr路由断言工厂

1) 规则

该断言工厂用于判断请求提交的所要访问的 IP 地址是否在断言中指定的 IP 范围。当请求中的 IP 在指定范围时，匹配成功，断言为 true。

2) 配置文件

```
1      #RemoteAddr断言工厂
2      - id: remoteAddr_route
3        uri: https://www.baidu.com
4        predicates:
5          - RemoteAddr=10.20.33.150/180
```

3) 配置类

直接修改路由方法。

```
1 //remoteAddr配置路由规则
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("remoteAddr_route", ps -> ps.remoteAddr("10.20.33.150/180")
6             .uri("https://www.baidu.com"))
7         .build();
8 }
```

5.5 网关过滤器Filter

5.5.1 什么是过滤工厂&过滤器？

过滤器允许以某种方式修改传入的 HTTP 请求或返回的 HTTP 响应。而过滤器作用域是某些特定路由。

过滤器分类

过滤器允许以某种方式修改传入的 HTTP 请求或返回的 HTTP 响应。Filter 根据其作用范围的不同，分为两种：局部过滤器与全局过滤器

- **局部过滤器**：应用于单个路由策略上，其功能仅对路由断言为 true 的请求起作用
- **全局过滤器**：应用于所有路由策略上，其功能也是仅对路由断言为 true 的请求起作用

Spring Cloud Gateway 包括许多内置的 GatewayFilter 工厂。

5.5.2 环境准备三步走

以下讲义中的代码：

- yml配置文件式：在 05-gateway-config-filter 工程上直接进行修改，即yml配置文件。
- API 配置类式：在05-gateway-api-filter 工程上直接进行修改，即启动类中设置。

1) 第一步：定义一个工程05-showinfo

该工程就是为了显示通过过滤工厂对请求处理后的结果的。

A、定义工程

就定义一个普通的 Spring Boot 工程，仅导入一个 Spring Web 依赖，即spring-boot-web-starter 依赖。

B、定义处理器

```

@RestController
@RequestMapping("/info")
public class SomeController {

    // 暂时还没有定义任务处理器方法

}

```

C、定义处理器

```
1 | server.port: 8888
```

2) 第二步：创建config工程05-gateway-config-filter

直接复制05-gateway-config工程，并将配置文件中原来设置的路由策略全部删除。后续对于 filter 的配置式设置，都在该工程中进行。

3) 第三步：创建API工程05-gateway-api-filter

直接复制 05-gateway-api 工程，并将启动类中原来设置的路由策略全部删除。后续对于 filter 的 API 式设置，都在该工程中进行。

5.5.3 PrefixPath 过滤器：添加前缀

1) 规则

该过滤器工厂会为指定的 URI 自动添加上一个指定的 URI 前缀。

2) 配置式

```

1 | # 添加前缀 过滤器
2 | - id: prefixPath_filter
3 |   uri: http://localhost:8080
4 |   predicates:
5 |     - Path=/**
6 |   filters:
7 |     - PrefixPath=/order

```

3) 配置类

直接修改路由方法。

```

1 | //prefixPath过滤工厂
2 | @Bean
3 | public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4 |     return builder.routes()
5 |         .route("prefixPath_filter", ps -> ps.path("/**")
6 |             .filters(fs -> fs.prefixPath("/order"))
7 |             .uri("http://localhost:8080"))
8 |         .build();
9 | }

```

5.5.4 StripPrefix 过滤器：去除前缀

1) 规则

该过滤器工厂会为指定的 URI 去掉指定长度的前缀。

2) 配置式

```
1      # 去除前缀 过滤器
2      - id: stripPrefix_filter
3        uri: http://localhost:8080
4        predicates:
5          - Path=/**
6        filters:
7          - StripPrefix=1
```

3) 配置类

直接修改路由方法。

```
1  //stripPrefix过滤工厂
2  @Bean
3  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4      return builder.routes()
5          .route("stripPrefix_filter", ps -> ps.path("/**")
6              .filters(fs -> fs.stripPrefix(1))
7              .uri("http://localhost:8080"))
8          .build();
9  }
```

5.5.5 扩展了解过滤器工厂

1、AddRequestParameter 过滤器

1) 规则

该过滤器工厂会对匹配上的请求添加指定的请求参数。

2) 配置文件

```
1      # AddRequestParameter 过滤器
2      - id: addRequestParameter_filter
3        uri: http://localhost:8888
4        predicates:
5          - Path=/**
6        filters:
7          - AddRequestParameter=color, blue
```

3) 配置式-修改05-showinfo工程处理器

在处理器中添加如下处理器方法

```
1 @GetMapping("/param")
2 public String payment(String color) {
3     return "Color: " + color;
4 }
```

4) 配置类

直接修改路由方法。

```
1 //addRequestParameter过滤工厂
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("addRequestParameter_filter", ps -> ps.path("/**")
6             .filters(fs -> fs.addRequestParameter("color", "hero-blue"))
7             .uri("http://localhost:8080"))
8         .build();
9 }
```

2、AddResponseHeader 过滤器

1) 规则

该过滤器工厂会给从网关返回的响应添加上指定的 header。

2) 配置文件

```
1 # AddResponseHeader 过滤器
2 - id: addResponseHeader_filter
3   uri: http://localhost:8080
4   predicates:
5     - Path=/**
6   filters:
7     - AddResponseHeader=X-Response-Red, Blue
```

3) 配置类

直接修改路由方法。

```

1 //addResponseHeader过滤工厂
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("addResponseHeader_filter", ps -> ps.path("/**")
6             .filters(fs -> fs.addResponseHeader("X-Response-Red", "blue"))
7             .uri("http://localhost:8080"))
8         .build();
9 }

```

3、AddRequestHeader过滤器

OAuth的鉴权的令牌token放到请求头中的

1) 规则

该过滤器工厂会对匹配上的请求添加指定的 header。

2) 配置文件

这里修改的是 09-gateway-filter-config 工程中的配置文件。

```

1 # AddRequestHeader 过滤器
2 - id: addRequestHeader_filter
3   uri: http://localhost:8080
4   predicates:
5     - Path=/**
6   filters:
7     - AddRequestHeader=X-Request-red, blue

```

3) 修改showinfo工程处理器

在处理器中添加如下处理器方法。

```

1 @RestController
2 @RequestMapping("info")
3 public class SomeController {
4     //暂时还没有定义任务处理器方法
5     @RequestMapping("header")
6     public String headerHandler(HttpServletRequest request){
7         String header = request.getHeader("X-Request-red");
8         return "X-Request-red: " + header;
9     }
10 }

```

4) 配置类

直接修改路由方法。

```

1 //addRequestHeader过滤器
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("addRequestHeader_filter", ps -> ps.path("/**")
6             .filters(fs -> fs.addRequestHeader("X-Request-red", "blue"))
7             .uri("http://localhost:8080"))
8         .build();
9 }

```

4、RewritePath 过滤器

1) 规则

该过滤器工厂会将请求 URI 替换为另一个指定的 URI 进行访问。RewritePath 有两个参数，第一个是正则表达式，第二个是要替换为目标表达式。

2) 配置文件

将 `http://hero.io/information/param/**` 这个路径重写为 `http://hero.io/info/param/**`

```

1 # 重写路径 过滤器
2 - id: rewritePath_filter
3   uri: http://localhost:8888
4   predicates:
5     - Path=/information/**
6   filters:
7     - RewritePath=/information(?<segment>/?.*), /info${segment}

```

如上RewritePath的含义：对于请求路径 /information，当前的配置在请求到达前会被重写为 /info，由于YAML的语法问题，\$符号后面应该加上\

想要理解上述正则表达式需要熟悉正则的两个概念：

1. 命名分组：`(?<name>capturing text)`：将匹配的子字符串捕获到一个组名称中，后面可通过分组名获得匹配结果。例如：这个示例就是将 capturing text 捕获到名称为 name 的组中
2. 引用捕获文本：`${name}`：将名称为name的命名分组对应的内容替换到此处

对于配置文件中的：`- RewritePath=/information(?<segment>/?.*), /info${segment}`
详解：

`(?<segment>/?.*)`：

- `?<segment>` 名称为 segment 的组
- `/?` 字符/出现0次或1次
- `.*` 任意字符出现0次或多次

合起来就是：将 `/?.*` 匹配到的结果捕获到名称为segment的组中，`${segment}`：将名称为 segment 的分组捕获到的文本置换到此处。

注意：\的出现是由于避免 yaml 语法认为这是一个变量（因为在 yaml 中变量的表示法为 `${variable}`，而这里我们想表达的是字面含义），在 gateway 进行解析时，会被替换为 `${segment}`

直接修改路由方法。

```
1 //rewritePath过滤工厂
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     return builder.routes()
5         .route("rewritePath_filter", ps -> ps.path("/**")
6             .filters(fs -> fs.rewritePath("/information(<segment>/?.*)",
7                 "/info${segment}"))
8             .uri("http://localhost:8888"))
9     .build();
10 }
```

5.5.6 自定义过滤器

前面的 GatewayFilter 工厂可以创建出某种特定的 Filter 过滤效果，但这些过滤功能可能并不能满足全部业务需求，此时可以根据具体需求自定义自己的 Filter。

1、自定义GatewayFilter：修改请求，向请求头添加一个参数。

需求：动态追加授权认证token

在05-gateway-api-filter工程中完成案例

1) 定义GatewayFilter

```
1 /**
2  * 在自定义的 Filter 中为请求添加指定的请求头。
3  */
4 public class AddHeaderGatewayFilter implements GatewayFilter {
5     @Override
6     public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
7         chain) {
8         //1. 构建改变后的Request对象
9         ServerHttpRequest request =
10             exchange.getRequest().mutate().header("X-Request-red", "blue").build();
11         //2. 将改变后的对象设置到exchange对象中
12         ServerWebExchange webExchange =
13             exchange.mutate().request(request).build();
14         //3. 设置修改后的exchange
15         return chain.filter(webExchange);
16     }
17 }
```

2) 修改启动类

仅修改路由方法。

```

1 //配置路由：自定义拦截器
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     //路由构建器对象，构建一个路由规则
5     return builder.routes().route("AddHeader_route",predicateSpec ->
6         predicateSpec
7             .path("/**")
8             .filters(gfs -> gfs.filter(new
9                 AddHeaderGatewayFilter()))
10             .uri("http://localhost:8888")).build();
11 }

```

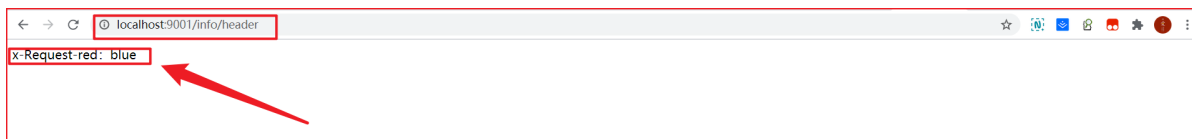
3) 修改ShowInfo工程的处理器

```

1 @RestController
2 @RequestMapping("info")
3 public class SomeController {
4     //暂时还没有定义任务处理器方法
5     @RequestMapping("header")
6     public String headerHandler(HttpServletRequest request){
7         String header = request.getHeader("x-Request-red");
8         return "x-Request-red: " + header;
9     }
10 }

```

4) 测试



2. 自定义GatewayFilter: 多filter

下面我们要定义出多个 Filter，每个 Filter 都具有 pre 与 post 两部分。将所有 Filter 注册到路由中，以查看它们执行的顺序。

需求：统计监控API接口调用的时间

1) 在工程05-gateway-filter-custom中定义三个GatewayFilter

A、第一个Filter

```

1 /**
2  * 目标：查看过滤器执行顺序，观察pre过滤和post过滤
3  */
4 //过滤器1
5 public class OneGatewayFilter implements GatewayFilter {
6     @Override
7     public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
8         chain) {
9         //获取系统当前时间
10         long startTime = System.currentTimeMillis();

```

```

10         System.out.println("pre-filter-【111】 " + startTime);
11         //设置filter过滤器时间
12         exchange.getAttributes().put("startTime", startTime);
13         return chain.filter(exchange).then(Mono.fromRunnable(() -> {
14             System.out.println("post-filter-【111】 ");
15             //获取过滤器执行开始时间
16             Long startTimeAttr = (Long)
exchange.getAttributes().get("startTime");
17             //获取过滤器执行结束时间
18             long endTime = System.currentTimeMillis();
19             //计算开始到结束时间差值
20             System.out.println("OneGatewayFilter过滤器执行用时: " + (endTime -
startTime));
21         }));
22     }
23 }
24

```

B、第二个Filter

```

1 //过滤器2
2 public class TwoGatewayFilter implements GatewayFilter {
3     @Override
4     public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
5         System.out.println("pre-filter-【222】 ");
6         return chain.filter(exchange).then(Mono.fromRunnable(() -> {
7             System.out.println("post-filter-【222】 ");
8         }));
9     }
10 }

```

C、第三个Filter

```

1 //过滤器3
2 public class ThreeGatewayFilter implements GatewayFilter {
3     @Override
4     public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
5         System.out.println("pre-filter-【333】 ");
6         return chain.filter(exchange).then(Mono.fromRunnable(() -> {
7             System.out.println("post-filter-【333】 ");
8         }));
9     }
10 }

```

2) 修改启动类，配置三个过滤器

仅修改路由方法。

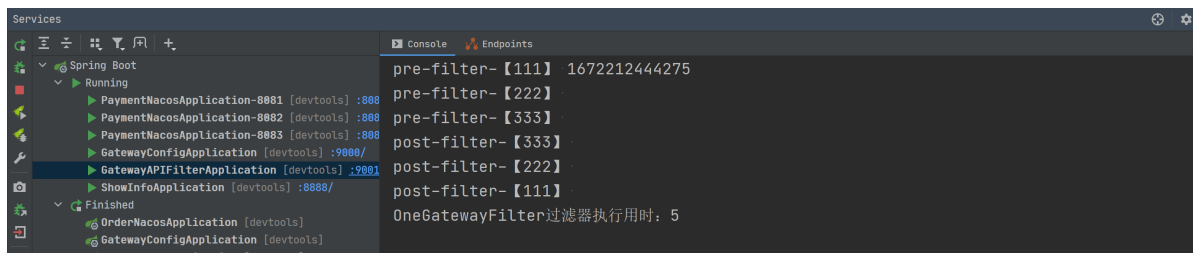
```
1 //配置三个拦截器
2 @Bean
3 public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4     //路由构建器对象，构建一个路由规则
5     return builder.routes()
6         .route("custom_filter_route", ps -> ps.path("/**")
7             .filters(gfs -> gfs
8                 .filter(new OneGatewayFilter())
9                 .filter(new TwoGatewayFilter())
10                .filter(new ThreeGatewayFilter()))
11         .uri("http://localhost:8888"))
12     .build();
13 }
```

3) 在ShowInfo工程的添加处理器

在 05-showinfo 工程中添加如下处理器。

```
1 //获取当前系统时间
2 @RequestMapping("time")
3 public String time(HttpServletRequest request){
4     return "time: " + System.currentTimeMillis();
5 }
```

4) 测试结果



3、自定义Global Filter：模拟网关鉴权

这里要实现的需求是，访问当前系统的任意模块的 URL 都需要是合法的 URL。这里所谓合法的 URL 指的是请求中携带了 token 请求参数。

由于是对所有请求的 URL 都要进行验证，所以这里就需要定义一个 Global Filter，可以应用到所有路由中。

注意：Global Filter 不需要在任何具体的路由规则中进行注册，只需在类上添加@Component注解，将其生命周期交给 Spring 容器来管理即可。

1) 在工程05-gateway-api-filter中定义GlobalFilter

```
1  /**
2   * 目标：模拟网关鉴权
3   */
4  @Component//注意：必须注入Spring容器，否则不能生效
5  public class URLValidateFilter implements GlobalFilter, Ordered {
6      @Override
7      public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain
chain) {
8          //1.获取请求参数token
9          String token =
exchange.getRequest().getQueryParams().getFirst("token");
10         //2.判断token是否存在
11         //如果不存在则拦截，提示用户未授权
12         if (StringUtils.isEmpty(token)){
13             //设置提示用户未授权
14             exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
15             //完结请求
16             return exchange.getResponse().setComplete();
17         }
18         //如果存在，则放行拦截器
19         return chain.filter(exchange);
20     }
21
22     @Override
23     public int getOrder() {
24         return Ordered.HIGHEST_PRECEDENCE;//最高优先级
25     }
26 }
```

3) 修改启动类

```
1  //配置全局权限校验过滤器
2  @Bean
3  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
4      //路由构建器对象，构建一个路由规则
5      return builder.routes()
6          .route("custom_global_filter_route",ps -> ps.path("/**")
7              .uri("http://localhost:8888"))
8          .build();
9  }
```

4) 测试: 设置token和不设token参数

Name	Status	Type	Initiator	Size	Time	Waterfall
time?token=12	200	document	Other	135 B	12 ms	
time	401	document	Other	48 B	4 ms	
data:image/png;base64...	200	png	chrome-error://chromewebdata/5309	(memory cache)	0 ms	
data:image/png;base64...	200	png	chrome-error://chromewebdata/5309	(memory cache)	0 ms	
data:image/png;base64...	200	png	chrome-error://chromewebdata/5309	(memory cache)	0 ms	

包含token可以访问成功，不包含则失败

4、过滤器的默认优先级：

- 局部 filter 的优先级要高于默认 Filter 的。
- 相同路由策略，配置式【配置文件】的要高于 API 式【配置类】的。

5、如何修改自定义过滤器执行顺序？

在自定义了一些 GlobalFilter 后，为了保证这些 Filter 的执行顺序，在每个返回 GlobalFilter 的 @Bean 方法上添加 @Order，或直接使自定义的 GlobalFilter 类实现 Order 接口，就可以修改 Filter 的执行顺序。

```
@Bean
@Order(-1)
public GlobalFilter myGlobalFilter() {
    return (exchange, chain) -> {
        Log.info("first pre filter");
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            Log.info("third post filter");
        }));
    };
}
```

```
@Component
public class URLValidateFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilt
```

5.6 网关服务降级：

实现需求：我们下面的例子实现的需求是，浏览器访问 05-showinfo 的处理器不成功，从而服务降级到 Gateway 工程中定义的降级处理器。

为什么需要降级呢？

- 提升微服务架构系统的容错性

当下游接口负载很大，或者接口不通等其他原因导致超时，如果接口不熔断的话将会影响到下游接口得不到喘息，网关也会因为超时连接一直挂起，很可能因为一个子系统的问题导致整个系统的雪崩。所以我们的网关需要设计熔断，当因为熔断器打开时，网关将返回一个降级的应答。

1) 添加依赖

05-gateway-config-filter 与 05-gateway-api-filter 两个工程中均需要添加上resilience4j 依赖。

```
1 <!--resilience4j 依赖-->
2 <dependency>
3   <groupId>org.springframework.cloud</groupId>
4   <artifactId>spring-cloud-starter-circuitbreaker-reactor-
    resilience4j</artifactId>
5 </dependency>
```

2) 定义降级处理器

05-gateway-config-filter 与 05-gateway-api-filter 两个工程中均需要添加上该降级处理器。

```
1 @RestController
2 public class GatewayFallbackController {
3     //定义服务降级处理方法
4     @RequestMapping("/fallback")
5     public Map<String,Object> defaultfallback(){
6         Map<String,Object> map = new HashMap<>();
7         map.put("code",200);
8         map.put("msg","服务超时降级");
9         map.put("data",null);
10        return map;
11    }
12 }
```

4) 配置文件

```
1 # 降级过滤器
2 - id: circuitBreaker_filter
3   uri: http://localhost:8888
4   predicates:
5     - Path=/**
6   filters:
7     - name: CircuitBreaker
8       args:
9         name: myCircuitBreaker
10        fallbackUri: forward:/fallback
```

5) 配置类

直接修改路由方法。

```
1  @Bean
2  public RouteLocator someRouteLocator(RouteLocatorBuilder builder) {
3      return builder.routes()
4          .route("circuitBreaker_filter", ps -> ps.path("/**")
5              .filters(fs -> fs.circuitBreaker(config -> {
6                  config.setName("myCircuitBreaker");
7                  config.setFallbackUri("forward:/fallback");
8              }))
9          .uri("http://localhost:8888"))
10     .build();
11 }
```

6) 测试熔断降级

1. 超时服务降级

```
1 | http://localhost:9000/info/time
```

返回

```
1 | {"msg":"服务超时降级","code":200,"data":null}
```

2. 其他异常

`spring-cloud-gateway` 调用下游服务返回的异常，网关不做任何处理，会直接返回。

7) 配置全局熔断降级

前面的 `GatewayFilter` 工厂是在某一特定路由策略中设置的，仅对这一种路由生效。若要使某些过滤效果应用到所有路由策略中，就可以将该 `GatewayFilter` 工厂定义在默认 `Filters` 中。修改 `gateway` 工程配置文件。

```
1  spring:
2    cloud:
3      gateway:
4        default-filters:
5          - name: CircuitBreaker
6            args:
7              name: myCircuitBreaker
8              fallbackUri: forward:/fallback
9        routes:
10         - id: test_filter
11           uri: http://localhost:8080
12           predicates:
13             - Path=/**
```


5.7 网关限流

配置网关限流

做人要有底线，做系统要有上线！

步骤：

1. 启动redis服务【令牌桶放到redis里面】
2. 导入依赖坐标
3. 修改配置

1) 规则

该过滤器工厂会对进来的请求进行限流。这里采用的是**令牌桶算法**。另外，从这里也可以看出其是基于 Redis 实现的，所以需要导入 Redis 的依赖。

令牌：ip地址、访问host名称....

2) 导入依赖

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
4 </dependency>
```

3) 修改启动类

```
1 @SpringBootApplication
2 public class GatewayConfigApplication {
3
4     public static void main(String[] args) {
5         SpringApplication.run(GatewayConfigApplication.class, args);
6     }
7
8     //配置令牌桶算法的key:将主机名称作为限流key
9     @Bean
10    public KeyResolver keyResolver(){
11        return exchange -> Mono.just(exchange
12            .getRequest()
13            .getRemoteAddress()
14            .getHostName());
15    }
16 }
```

4) 配置文件

```
1      # 限流过滤器
2      - id: requestRateLimiter_filter
3        uri: http://localhost:8888
4        predicates:
5          - Path=/**
6        filters:
7          - name: RequestRateLimiter
8            args:
9              key-resolver: "#{@keyResolver}"
10             redis-rate-limiter.replenishRate: 10
11             redis-rate-limiter.burstCapacity: 50
```

解释：

- burstCapacity：令牌桶总容量。
- replenishRate：令牌桶每秒填充平均速率。
- key-resolver：用于限流的键的解析器的 Bean 对象的名字。它使用 SpEL 表达式根据#{@beanName}从 Spring 容器中获取 Bean 对象。

通过在 replenishRate 和中设置相同的值来实现稳定的速率 burstCapacity。设置 burstCapacity 高于时，可以允许临时突发 replenishRate。在这种情况下，需要在突发之间允许速率限制器一段时间（根据 replenishRate），因为2次连续突发将导致请求被丢弃（HTTP 429 - Too Many Requests）

key-resolver: "#{@userKeyResolver}" 用于通过SPEL表达式来指定使用哪一个KeyResolver.

如上配置 表示 1 秒内，允许 10 个请求通过，令牌桶的填充速率也是1秒钟添加10个令牌。

最大突发状况 允许 1 秒内有50 次请求。

5) 测试步骤

- 启动本地redis
- 启动gateway网关
- 打开浏览器 <http://localhost:9000/info/time>
- 快速刷新，当1秒内发送多次请求，就会返回429错误。

6) 测试

注意：需要启动Redis的服务

